OCS INTELLIGENCE · SPRINT 4 · PLAYBOOK

Inference on both rigs, behind one chat

Rig 1 and Rig 2 both serve models, and ai.opencodingsociety.com shows all three in one model picker.

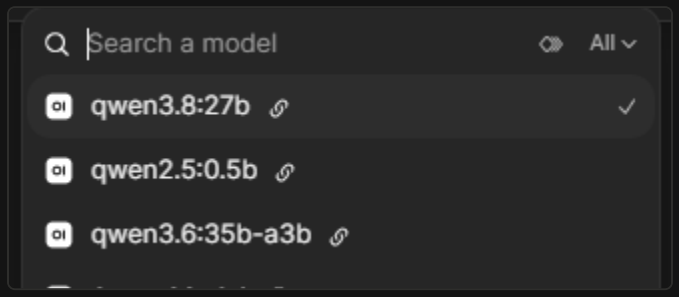
STATE **done** | DATE 2026-10-15 | PROJECT PROMX-OCS

SOURCE evidence/sprint4/2026-10-15-inference-on-both-rigs.md

Sprint 4's goal is inference on both rigs, offered through one provider. The first half is done. Rig 2's worker was crash-looping until 2026-10-15, when we fixed it, gave Rig 2 its own orchestrator, and moved it to a faster mixture-of-experts model. We then connected Open WebUI to it as a second connection. On 2026-10-16 the model picker at `https://ai.opencodingsociety.com/` showed all three models to a regular user. The top two are served by Rig 1 and the bottom one by Rig 2. Sprint 4 closes here, on 2026-10-23: both rigs serve, and students reach every model from one chat. Putting both rigs behind **one** API, so Open WebUI needs one connection and Rig 2 goes through the EC2 wait line, moves to Sprint 5, which is about the orchestrator (roadmap).

At a glance

Goal	Both rigs serve inference through one provider
Outcome	Both rigs serve. One Open WebUI shows every model; users pick a model, not a rig. Since 2026-10-19, Rig 1 uses all 8 cards and Rig 2 all 9 (step 7)
Moved to Sprint 5	One router that knows both rigs, so the public `/v1` API and Open WebUI both see one provider
Where	Rig 2 (`100.75.127.192`): worker, orchestrator. Rig 1: Open WebUI container. NetBird: `rigs-to-rigs`
Why	Decision 0009, **proposed**



The model picker on ai.opencodingsociety.com, seen as a regular user on 2026-10-16. qwen3.8:27b and qwen2.5:0.5b come from Rig 1; qwen3.6:35b-a3b comes from Rig 2.

MODEL	RIG	PATH FROM OPEN WEBUI	GENERATION, ONE REQUEST
`qwen3.8:27b`	Rig 1, Worker A (5 GPUs)	connection 0: EC2 gateway → wait line → Rig 1 orchestrator	~8.7 tok/s
`qwen2.5:0.5b`	Rig 1, Worker B (2 GPUs)	connection 0: same path	~80–97 tok/s
`qwen3.6:35b-a3b`	Rig 2, all 8 GPUs	connection 1: straight to Rig 2's orchestrator over NetBird	~27 tok/s

This is the layout on 2026-10-16. Speeds come from single requests (Rig 1 log, Rig 2 record). They aren't benchmarks. The layout changed afterwards; see step 7.

Before

Rig 2 came back from the Sprint 3 rebuild with `infra/rig2/setup.sh` already run: CUDA 12.6, llama.cpp, the model, and `ocs-llama.service` were all in place. But it served nothing:

CHECK	WHAT WE SAW
`ocs-llama.service`	Crash-looping, restart counter at **4,531**
Worker log	`couldn't bind HTTP server socket, hostname: 127.0.0.1, port: 8081`
What held `:8081`	A placeholder `python3 -m http.server 8081 --bind 0.0.0.0`, started from a desktop terminal for a planned `ai2.opencodingsociety.com` page
GPU memory	7 MiB per card: no model loaded

The plan at the time was a separate public site for Rig 2. The team decided against it: there should be **one** Open WebUI, with a router choosing the rig.

What we did

All of this was done over NetBird SSH from an `outbound-only` operator peer (NetBird playbook).

1. **Free the port and restart the worker.** We killed the placeholder (its folder was left in place) and ran `systemctl restart ocs-llama`. The worker was healthy about 2 minutes later, serving `qwen3.8:27b` at **8.8 tok/s**. That's the same speed as Rig 1's Worker A, with 8 GPUs instead of 5.
2. **Give Rig 2 its own orchestrator.** We deployed this repo's `orchestrator/` to `/opt/ocs-orchestrator/`, with Worker B made optional (Rig 2 runs one worker). Unlike Rig 1, Rig 2 can reach PyPI, so the venv came from PyPI. Config: `infra/rig2/orchestrator.env.template` and `infra/rig2/ocs-orchestrator.service`.

```
/readyz                                {"status":"ready"}
/v1/models, wrong key                  401
/v1/chat/completions model=qwen2.5:0.5b  404
```

3. **Switch Rig 2 to a mixture-of-experts model.** Every card is on a PCIe gen 1 ×1 riser, and layer split runs one GPU at a time, so more GPUs add memory, not speed. A dense model reads all of its weights for every token. **Qwen3.6-35B-A3B** activates only ~3B of its 35B parameters per token. We downloaded Unsloth's `UD-Q4_K_XL` GGUF (22.4 GB, SHA-256 checked against Hugging Face) and changed `worker.env`:

```
DEFAULT_MODEL_ID=qwen3.6:35b-a3b
TENSOR_SPLIT=11,8,8,8,8,8,8,8   # GPU 0 is the 11 GB 1080 Ti
PARALLEL=2
CONTEXT_SIZE=524288             # 2 slots x the model's full 262,144 tokens
LLAMA_EXTRA_ARGS=-ub 1024
```

   Generation went from 8.8 to **27.2 tok/s**. This config was picked without benchmarking, at the team's request.

4. **Let the rigs talk.** Open WebUI runs on Rig 1, so it needs Rig 2's `:9000`. We added the temporary `rigs-to-rigs` NetBird policy (commit `c2ac00d`).

5. **Add Rig 2 to Open WebUI as a second connection.** We generated a dedicated key for Open WebUI → Rig 2 and appended it to Rig 2's `PUBLIC_API_KEYS`. We backed up Open WebUI's database (`webui.db.bak-20261015`), added connection 1, `http://100.75.127.192:9000/v1`, and restarted the container. From inside the container, both connections listed their models, and a chat through connection 1 ran at 27.8 tok/s.

6. **Check it in a browser** (2026-10-16). Signed in as a regular user, the picker shows `qwen3.8:27b`, `qwen2.5:0.5b`, and `qwen3.6:35b-a3b` (screenshot above).

7. **What changed by 2026-10-19.** Checked over SSH (capture). The hardware side is its own playbook: a ninth GPU, a driver that broke.

WORKER	2026-10-16	2026-10-19
Rig 1, Worker A	`qwen3.8:27b` on 5 GPUs	`qwen3.8:27b` on 4 GPUs
Rig 1, Worker B	`qwen2.5:0.5b` on 2 GPUs	`gemma4:26b` on 4 GPUs (config dated 2026-10-16; who changed it isn't recorded)
Rig 2	`qwen3.6:35b-a3b` on 8 GPUs	Same model on 9 GPUs, 27.6–27.8 tok/s

   Rig 1's eighth card, an open question when this playbook started, is in use.

> **IMPORTANT**
>
> Open WebUI 0.11.3 merges the model lists of its connections, but if two connections offer the **same model ID** it keeps only the first. That's why Rig 2 serves a *different* model instead of a second copy of `qwen3.8:27b`: two rigs with the same model name would never share load through Open WebUI.

How to check it

CHECK	PASS LOOKS LIKE	IF IT FAILS
Model picker on `https://ai.opencodingsociety.com/`, as a regular user	Three models, as in the screenshot	Only two: connection 1 is off, or Rig 2's orchestrator is down
On Rig 2: `systemctl is-active ocs-llama ocs-orchestrator`	`active` twice	`journalctl -u <unit> -n 80`. Check whether something else holds `:8081` (`ss -tlnp`)
On Rig 2: `curl -s http://127.0.0.1:9000/readyz`	`{"status":"ready"}`	The worker is still loading (about 2 min) or has crashed
From the Rig 1 host: `nc -vz 100.75.127.192 9000`	`succeeded`	The `rigs-to-rigs` policy is missing: `python3 infra/netbird/apply.py check`
A short chat with `qwen3.6:35b-a3b`	An answer at about 27 tok/s	Empty `content` with a long `reasoning_content` means `max_tokens` ran out while the model was thinking

Pitfalls

SYMPTOM	CAUSE	FIX
Worker restart counter in the thousands, `couldn't bind ... 8081`	Another process (a test web server) held the port on `0.0.0.0`	`ss -tlnp | grep 8081`, stop the other process. Keep test servers off worker ports
A second rig serving the same model never gets traffic in Open WebUI	Open WebUI keeps only the first connection that offers a model ID	Serve different model names, or put a router in front
Rig 2 orchestrator accepts any Bearer token	`PUBLIC_API_KEYS` left empty	Always set it. Rig 2 has a temporary operator key and the Open WebUI key, not the student key
First chat after a restart returns a body without `choices`	Not captured. The next request worked	Retry once. If it repeats, capture the body
Short answers come back empty	The model thinks first; `max_tokens` spent on `reasoning_content`	Use a higher `max_tokens`

Open items

Sprint 4 is done. These carry over.

- **One provider** (Sprint 5, orchestrator). Put a router that knows both rigs in front of them: the planned Mac minis, or EC2's gateway in the meantime. Then the public `/v1` API serves Rig 2's model too, Open WebUI goes back to one connection, and `rigs-to-rigs` can be removed (Sprint 5 plan).
- **Rig 2 skips the wait line** until then. Open WebUI → Rig 2 goes straight to the orchestrator. Only its reject threshold (16) and llama-server's 2 slots limit load.

- **Benchmark both rigs' configs** (Sprint 5, benchmarking): Rig 2 against fewer GPUs, `PARALLEL`, `-ub`; Rig 1's new 4 + 4 split; `gemma4:26b`'s speed. A `llama-bench` run on Rig 2 was started and interrupted before it printed anything.
- **Decide on 0009.** It's `proposed`. It also differs from the blueprint, which has Rig 2 as dev/test, not serving students.
- **Leftovers from the `ai2` attempt:** a TLS cert and Nginx site on EC2 (answering `301` to itself), and `/home/ocs/ai2-site` on Rig 2.

Sources

- Rig 2: worker crash loop fixed, orchestrator deployed, switched to a MoE model
- Open WebUI connected to Rig 2
- Decision 0009
- Model picker screenshot: taken by the team on 2026-10-16, signed in as a regular user
- Rig 2: ninth GPU, driver 580, for step 7
- Commits `3319c30` (Rig 2 inference), `598b477` (Open WebUI connection), `952e972` (docs), `c2ac00d` (`rigs-to-rigs`)